

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

ЛАБОРАТОРНАЯ РАБОТА №2

Работа с базами данных и ORM Sequelize

Тема проекта: платформа для обмена игровыми скриншотами и видео

Студент: _____

Группа: _____

Преподаватель: _____

Минск, 2026

1. Цель работы

Освоить подключение PostgreSQL к Node.js-приложению, использовать ORM Sequelize для создания модели и CRUD-операций, выполнить миграцию изменения схемы и наполнить базу тестовыми данными.

2. Краткие теоретические сведения

ORM (Object-Relational Mapping) связывает таблицы реляционной базы данных с объектами программы. Вместо ручного SQL разработчик вызывает методы модели: create, findAll, findByPk, update и destroy. Sequelize является ORM для Node.js и поддерживает PostgreSQL, миграции, валидацию и связи между моделями.

Преимущества ORM: меньше повторяющегося SQL, единый стиль работы с данными и переносимость схемы через миграции. Недостатки: дополнительный уровень абстракции и возможность получить неоптимальный запрос при сложной выборке.

3. Настройка базы данных

Создан бесплатный облачный проект Supabase «Game Media API» в регионе Central EU (Frankfurt). Supabase предоставляет полноценную PostgreSQL-базу. Приложение подключается через Session pooler по защищённому SSL-соединению. Строка подключения хранится только в файле .env, который исключён из Git.

Конфигурация Sequelize

```
require('dotenv').config();

const databaseConfig = {
  use_env_variable: 'DATABASE_URL',
  dialect: 'postgres',
  dialectOptions: {
    ssl: { require: true, rejectUnauthorized: false },
  },
  logging: false,
};
```

Установленные зависимости

```
npm install sequelize pg pg-hstore dotenv
npm install --save-dev sequelize-cli
```

4. Модель Post

Модель продолжает предметную область первой лабораторной. Поле comments хранится как PostgreSQL JSONB, а обязательные строки проверяются средствами Sequelize и контроллера.

```
Post.init({
  title: { type: DataTypes.STRING, allowNull: false },
  game: { type: DataTypes.STRING, allowNull: false },
  mediaType: {
    type: DataTypes.STRING, allowNull: false,
    validate: { isIn: [['screenshot', 'video']] },
  },
  mediaUrl: { type: DataTypes.STRING, allowNull: false },
  description: { type: DataTypes.TEXT, allowNull: false },
  comments: { type: DataTypes.JSONB, defaultValue: [] },
  likesCount: { type: DataTypes.INTEGER, defaultValue: 0 },
}, { sequelize, modelName: 'Post' });
```

5. Структура таблицы Posts

Колонка	Тип PostgreSQL	NULL
id	integer	нет
title	varchar	нет
game	varchar	нет
mediaType	varchar	нет
mediaUrl	varchar	нет
description	text	нет
comments	jsonb	нет
createdAt	timestampz	нет
updatedAt	timestampz	нет
likesCount	integer	нет

Структура получена запросом к `information_schema` после выполнения миграций в Supabase. Поле `likesCount` присутствует в таблице и демонстрирует изменение схемы отдельной миграцией.

6. Миграции и seed

Первая миграция создаёт таблицу `Posts`. Вторая миграция изменяет уже существующую схему и добавляет новое поле:

```
async up(queryInterface, Sequelize) {
  await queryInterface.addColumn('Posts', 'likesCount', {
    type: Sequelize.INTEGER,
    allowNull: false,
    defaultValue: 0,
  });
}

async down(queryInterface) {
  await queryInterface.removeColumn('Posts', 'likesCount');
}
```

Миграции и seed выполнены командами:

```
npm run db:migrate
npm run db:seed

20260921204129-create-post: migrated
20260921204206-add-likes-count-to-posts: migrated
20260921204214-demo-posts: migrated
```

Seed добавляет две записи: скриншот из Cyberpunk 2077 и видео из Rocket League.

7. Реализация CRUD

Метод	URL	Sequelize	Код
GET	/posts	<code>Post.findAll()</code>	200
GET	/posts/:id	<code>Post.findByPk(id)</code>	200 / 404
POST	/posts	<code>Post.create(data)</code>	201 / 400
PUT	/posts/:id	<code>post.update(data)</code>	200 / 400 / 404
DELETE	/posts/:id	<code>Post.destroy({ where })</code>	204 / 404

Ключевой фрагмент контроллера

```
const posts = await Post.findAll({ order: [['id', 'ASC']] });
const post = await Post.findByPk(req.params.id);
const newPost = await Post.create(req.body);
await post.update(req.body);
await Post.destroy({ where: { id: req.params.id } });
```

8. Тестирование API

Сервер запущен на <http://localhost:3001>. Запросы выполнены после миграций и seed непосредственно к базе Supabase. Для демонстрации подготовлена коллекция Postman «Lab 2 - Sequelize».

Запрос	Статус	Результат
GET /posts	200 OK	Получены две seed-записи из PostgreSQL
POST /posts	201 Created	Создана публикация Boss fight
GET /posts/:id	200 OK	Созданная запись найдена по первичному ключу
PUT /posts/:id	200 OK	Поля публикации и likesCount обновлены
DELETE /posts/:id	204 No Content	Запись удалена из PostgreSQL
GET удалённой записи	404 Not Found	{"error": "Post not found"}
POST с неверными данными	400 Bad Request	Ошибка обязательного поля title

Пример тела POST

```
{
  "title": "Boss fight",
  "game": "Elden Ring",
  "mediaType": "video",
  "mediaUrl": "https://example.com/boss.mp4",
  "description": "Final boss battle",
  "comments": ["Amazing!"],
  "likesCount": 3
}
```

9. Ответы на контрольные вопросы

1. ORM

Сопоставляет таблицы с объектами. Упрощает CRUD и миграции, но добавляет абстракцию и может создавать неоптимальные запросы.

2. Названия таблиц

Модель Post по умолчанию использует таблицу Posts. Поведение меняют tableName и freezeTableName.

3. WHERE

`Post.findAll({ where: { mediaType: 'video' } })`.

4. Миграции

Версионировать изменения схемы. up применяет изменение, down отменяет его.

5. Один-ко-многим

`User.hasMany(Post)` и `Post.belongsTo(User)`; внешний ключ userId хранится в Posts.

6. Пагинация

`Post.findAll({ limit: 10, offset: 20 })`.

7. Уникальность

Перехватить `SequelizeUniqueConstraintError` и вернуть 400 или 409.

8. `findByPk` и `findOne`

`findByPk` ищет по первичному ключу, `findOne` - первую запись по произвольному `where`.

9. Кастомный метод

Добавить метод экземпляра или `static`-метод в класс модели.

10. Связи

`belongsTo`, `hasOne`, `hasMany` и `belongsToMany`.

10. Вывод

В лабораторной работе временный массив первой работы заменён постоянным хранением в PostgreSQL Supabase. Sequelize подключён к Express-приложению, модель `Post` создана через CLI, CRUD переведён на асинхронные методы ORM. Выполнены две миграции, включая добавление поля `likesCount`, и seed начальных данных. Все основные и ошибочные сценарии API успешно проверены.